

Les Tests d'Intégration en Automatisation d'Infrastructure

Ce document est sur la conception et la mise en œuvre de **tests d'intégration** pour les modules et scripts d'infrastructure PowerShell. Il détaille la frontière entre tests unitaires et tests d'intégration, les techniques de provisionnement éphémère (*Sandboxing*), la gestion du cycle de vie des ressources de test (*Setup / Teardown*), le découplage via les tags Pester, ainsi que leur intégration dans une chaîne de déploiement continu (CI/CD).

1. La Pertinence des Tests d'Intégration en Infrastructure

Après avoir écrit des tests unitaires avec Pester où tout composant externe est simulé par un Mock, une question légitime se pose :

« Si 100 % de mes commandes sont mockées, comment puis-je être certain que mon code fonctionnera réellement face au véritable système d'exploitation, à Active Directory ou à ma base de données ? »

C'est précisément le rôle des **tests d'intégration**.

1.1 Comparatif : Tests Unitaires vs Tests d'Intégration

[TEST UNITAIRE (Isolation Totale)]

Script PowerShell —► [Mock Get-CimInstance] —► Réponse factice immédiate
(Zéro réseau, zéro écriture disque, vérifie l'algorithme)

[TEST D'INTÉGRATION (Validation des Flux Réels)]

Script PowerShell —► [Vraie Commande] —► [Vrai Service / Vrai Disque / Vraie API]
(Vérifie les permissions, la sérialisation, les schémas et les pilotes)

Critère d'évaluation	Test Unitaire (Unit Test)	Test d'Intégration (Integration Test)
Objectif	Valider la logique interne (conditions, boucles, pipeline).	Valider le dialogue réel avec les composants externes.
Emploi des Mocks	Systématique (zéro impact	Strictement proscrit sur le

	matériel ou réseau).	composant cible testé.
Vitesse d'exécution	Ultra-rapide ($\approx$ par test).	Plus lent ($\approx$ à plusieurs secondes par test).
Environnement requis	Le simple interpréteur PowerShell suffit.	Une machine avec l'environnement cible (droits admin, réseau, base de test).
Fragilité (<i>Flakiness</i>)	Nulle (résultats déterministes).	Modérée (sensible aux timeouts réseau, verrous de fichiers, droits).

1.2 Ce qu'un test unitaire ne détectera pas :

1. **Les incohérences de schémas ou de versions** : Votre mock renvoie une propriété FreeSpace, mais sur un vieux Windows Server, la propriété WMI s'appelle différemment.
2. **Les problèmes de sécurité et d'ACLs** : Votre algorithme est parfait sur le papier, mais l'écriture réelle sur le disque échoue en raison de restrictions NTFS ou de droits UAC insuffisants.
3. **Les erreurs de syntaxe SQL / WQL / REST** : Votre mock accepte n'importe quelle requête, mais le vrai serveur SQL rejette la requête pour une virgule manquante.
4. **Les verrous de fichiers en écriture concurrente** : Un autre processus ou l'antivirus bloque le fichier au moment précis où PowerShell tente de le manipuler.

2. L'Organisation des Tests au sein du Module

Pour que la campagne de tests reste rapide au quotidien pendant que le développeur écrit son code, on sépare physiquement les tests unitaires des tests d'intégration au sein de l'arborescence du module :

```

InfraTools/
├── InfraTools.psd1
├── InfraTools.psm1
├── Public/
│   ├── Set-CorporateAuditShare.ps1
│   └── Get-InfraDisque.ps1
├── Private/
│   └── Write-JournalModule.ps1
└── Tests/
    ├── Unit/
    └── # Tests rapides avec Mock (exécutés à chaque sauvegarde)
```

```

|   |   | Get-InfraDisque.Tests.ps1
|   |   | Write-JournalModule.Tests.ps1
|   |   | Integration/           # Tests réels d'infrastructure (exécutés avant commit / en CI)
|   |   | Set-CorporateAuditShare.Integration.Tests.ps1

```

Le Découplage par les Tags Pester 5 (-Tag)

Pester permet de classer les blocs Describe via des métadonnées appelées **Tags**. Cela permet d'exécuter soit uniquement les tests unitaires, soit uniquement les tests d'intégration :

```

Describe "Set-CorporateAuditShare - Validation Système Réelle" -Tag "Integration" {
    # Tests interagissant avec le vrai système
}

```

Commandes d'exécution ciblées en console :

1. Exécution quotidienne rapide (Uniquement les tests unitaires, sans toucher au système) :
Invoke-Pester -TagFilter "Unit"

2. Exécution lourde de validation (Uniquement les tests d'intégration réels) :
Invoke-Pester -TagFilter "Integration"

3. Le Défi Majeur : L'Hygiène et l'Idempotence (Teardown)

En test d'intégration d'infrastructure, le plus grand danger est la **pollution résiduelle** :

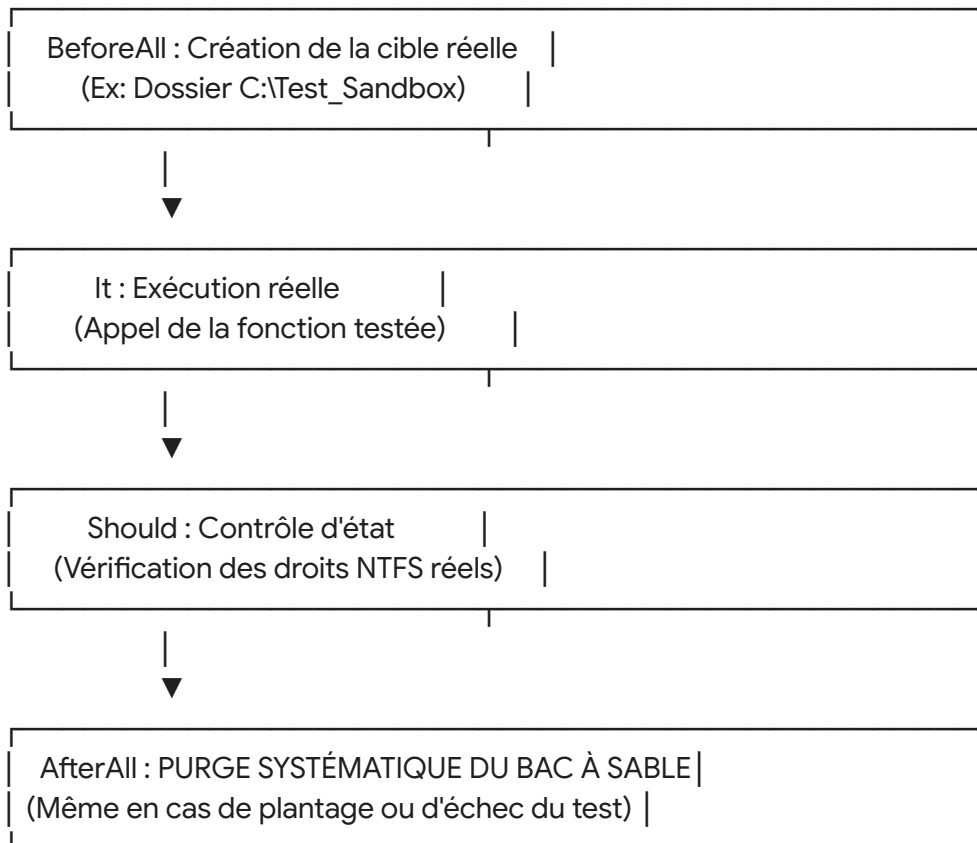
Si votre test crée un vrai dossier C:\Audit_Test, modifie une vraie clé de registre ou crée un compte utilisateur de test, que se passe-t-il si le test plante au milieu ?

Si la ressource n'est pas nettoyée :

- Les exécutions suivantes échoueront car « *la ressource existe déjà* ».
- Le serveur de test s'encombre de résidus de tests orphelins.

Le Bloc AfterAll Garanti

On applique rigoureusement le cycle de vie **Arrange / Act / Assert / Cleanup** grâce aux blocs BeforeAll et AfterAll :



4. Les Stratégies (*Sandboxing*)

Pour exécuter de vraies commandes sans mettre en danger l'infrastructure de production, on recourt à 3 techniques de confinement.

4.1 La Forteresse Locale Éphémère (Fichiers & Registre)

Plutôt que de tester sur des répertoires de production (C:\Windows, C:\Audit), le test crée son propre bac à sable identifié par un préfixe unique ou un GUID dans un répertoire temporaire réel :

- **Pour le système de fichiers :** Utiliser %env:TEMP ou une sous-arborescence éphémère (ex: C:\Test_Sandbox_Guid\).
(Attention : %TestDrive de Pester est pratique pour de la manipulation de fichiers simples, mais pour tester de véritables droits NTFS avancés, des partages SMB réels ou des verrous systèmes, un vrai chemin physique sur une partition NTFS réelle est requis).
- **Pour le Registre Windows :** Ne jamais écrire sous HKLM:\SOFTWARE\Microsoft. Créer une branche temporaire de test isolée (ex : HKLM:\SOFTWARE\Test_Validation_Suite).

4.2 Les Conteneurs Éphémères (Docker / Podman)

Pour tester des interactions avec une base de données SQL, un annuaire LDAP ou une API REST :

1. Pester démarre un conteneur léger en BeforeAll (ex: conteneur SQL Server ou OpenLDAP).
2. Les fonctions du module PowerShell dialoguent avec le conteneur via son port mappé.
3. Le conteneur est détruit en AfterAll.

4.3 Le Nommage Synthétique Canari

Si vous devez tester la création de ressources Active Directory ou de comptes de service sur un domaine de pré-production :

- Adopter une convention de nommage impossible à confondre avec la production : TEST_CANARI_20260910_XXXX.
- Placer systématiquement toutes les créations dans une **Unité d'Organisation (OU) dédiée aux tests automatisés**, soumise à une purge programmée automatique.

5. Cas Pratique d'Entreprise : Conception d'un Test d'Intégration

Considérons une fonction de production Set-SecuredLocalShare dont le rôle est :

1. De créer un répertoire sur le disque local.
2. De configurer des permissions NTFS strictes (suppression de l'héritage, octroi du contrôle total aux Administrateurs).
3. De publier le dossier sous forme de partage SMB avec un commentaire de conformité.

Le Fichier de Test d'Intégration :

Set-SecuredLocalShare.Integration.Tests.ps1

Dans ce test, **aucun Mock n'est utilisé** : le test crée physiquement le dossier et le partage sur l'OS, vérifie leur existence et leurs droits réels, puis détruit tout à la fin.

```
Describe "Tests d'Intégration : Set-SecuredLocalShare" -Tag "Integration" {  
    BeforeAll {  
        # 1. Préparation de l'environnement de bac à sable réel  
        $TestGuid = [guid]::NewGuid().ToString().Substring(0, 8)  
        $NomPartage = "TestShare_$TestGuid"  
        $DossierCible = Join-Path -Path $env:TEMP -ChildPath "SecuredShare_$TestGuid"  
  
        # Import forcé du module testé  
        Import-Module -Name ".\InfraTools.psd1" -Force  
    }  
}
```

```

Context "Validation du cycle de déploiement réel sur l'OS" {
  It "Doit créer le dossier physique et appliquer la protection NTFS réelle" {
    # Exécution de la commande réelle
    $Resultat = Set-SecuredLocalShare -NomPartage $NomPartage `
      -CheminDossier $DossierCible `
      -Description "Partage de test d'intégration"

    # 1. Vérification que le dossier physique existe réellement
    Test-Path -Path $DossierCible | Should -Be True

    # 2. Vérification de l'état réel de l'objet retourné
    $Resultat.Statut | Should -Be "ACTIF"
    $Resultat.Chemin | Should -Be $DossierCible

    # 3. Vérification de la configuration réelle du partage SMB sur l'OS
    $PartageSMB = Get-SmbShare -Name $NomPartage -ErrorAction SilentlyContinue
    $PartageSMB | Should -Not -BeNullOrEmpty
    $PartageSMB.Path | Should -Be $DossierCible
    $PartageSMB.Description | Should -Be "Partage de test d'intégration"
  }

  It "Doit être idempotent : un second appel avec la même cible ne doit pas lever d'erreur" {
    # Idempotence : réexécuter la commande sur une ressource déjà existante
    {
      Set-SecuredLocalShare -NomPartage $NomPartage `
        -CheminDossier $DossierCible `
        -Description "Partage de test d'intégration"
    } | Should -Not -Throw
  }
}

AfterAll {
  # 2. Nettoyage absolu (Teardown) pour laisser le poste 100 % propre
  Write-Verbose "Clôture des tests d'intégration : purge des ressources temporaires..."

  # Purge du partage SMB réel
  if (Get-SmbShare -Name $NomPartage -ErrorAction SilentlyContinue) {
    Remove-SmbShare -Name $NomPartage -Force -Confirm:$false
  }

  # Purge du dossier physique réel
  if (Test-Path -Path $DossierCible) {
    Remove-Item -Path $DossierCible -Recurse -Force -ErrorAction SilentlyContinue
  }
}

```

```
}  
}  
}
```

6. Bonnes Pratiques en Environnement d'Entreprise

1. Ne jamais exécuter de tests d'intégration sur un contrôleur de domaine de production :

Les tests d'intégration d'infrastructure doivent s'exécuter sur un poste de développement dédié, une machine virtuelle éphémère (Hyper-V / VMware) ou un lab d'intégration continue.

2. Exécuter les tests avec des privilèges adaptés :

La plupart des tests d'intégration interagissent avec le noyau Windows (services, partages, firewall). Ils doivent s'exécuter au sein d'une invite PowerShell ouverte avec les privilèges administrateur local.

3. Le principe du nettoyage inconditionnel :

Le bloc AfterAll doit tester l'existence de chaque ressource avant de la supprimer, et utiliser des clauses -ErrorAction SilentlyContinue pour qu'un échec de nettoyage ne masque pas les résultats des tests.

4. Isoler les tests dans la chaîne CI/CD :

Dans un pipeline GitLab CI ou GitHub Actions :

- Le premier job (*Lint & Unit*) exécute Invoke-Pester -TagFilter "Unit" sur n'importe quel runner rapide (Linux ou Windows conteneurisé).
- Le second job (*Integration*) déploie une machine virtuelle éphémère Windows Server pour exécuter Invoke-Pester -TagFilter "Integration".